

Detection and Classification of Radar and Sonar Pulses Using a CNN-LSTM Encoder-Decoder

Achraf Atifi
Department of Physics, University of Oslo

 <https://github.com/achat97/FYS5429>
(Dated: June 7, 2026)

We present a CNN–LSTM encoder–decoder network that detects acoustic pulses in a spectrogram and jointly estimates their type, timing, and frequency content. Casting the problem as a sequence-to-sequence task lets the model handle a variable and unknown number of pulses, providing a single, fully learned alternative to the matched filter. The network is trained on synthetic continuous-wave and frequency-modulated sonar pulses, subject to modeled transmission loss and reverberation, overlaid on real ocean noise. On held-out synthetic data it recovers the correct number of pulses in almost all cases and detects essentially every pulse with very few false alarms ($F_1 = 0.998$), while estimating the pulse times and frequencies with small, largely unbiased errors. Continuous-wave pulses are classified almost perfectly. The principal weakness is the discrimination between linear and hyperbolic sweeps, which are frequently confused because a short, narrowband hyperbolic sweep is nearly linear and so hard to distinguish in the time–frequency image.

CONTENTS

I. INTRODUCTION

I. Introduction	1
II. Theory	2
A. The underwater acoustic environment	2
B. Convolutional neural networks	3
C. Recurrent neural networks and the LSTM	3
D. Sequence-to-sequence encoder–decoder models	4
E. Evaluation metrics	4
III. Method	5
A. Dataset generation	5
B. Model architecture	6
C. Training and evaluation	6
IV. Results	7
A. Training behaviour	7
B. Pulse count	7
C. Detection	7
D. Classification	7
E. Regression	8
V. Discussion	8
A. Confusion between the swept types	9
B. Detection and localization	9
C. Limitations	9
D. Future work	10
VI. Conclusion	10
References	10

Radar and sonar estimate an object’s distance and velocity by utilizing the round-trip time and the Doppler-shifted frequency of the emitted wave. The traditional method to determine the two parameters is by the use of a matched filter, i.e., the echo is compared to several delayed and Doppler-shifted copies of the transmitted wave pulse. The result can then be represented in a 2-D format, where the peak lies at the correct delay and Doppler shift. The shape of this representation is given by the so-called ambiguity function, which is unique to each pulse type. The most common pulses are continuous wave (CW), linear frequency-modulated (LFM), and hyperbolic frequency-modulated (HFM) pulses. The ideal ambiguity function would have a single sharp peak at the correct delay and Doppler shift. In practice this is unattainable, which is why various pulse types, each suited to different objectives, are used [3, 9].

In this project, we present an alternative way to determine the temporal and frequency location of a pulse, as well as its type. Unlike the matched filter, which requires a bank of reference replicas and a separate procedure to identify the pulse type, our approach estimates the temporal and frequency location jointly with the pulse type in a single model, and learns directly from realistic ocean noise.

The problem at hand resembles acoustic classification, where several studies have applied deep learning using a CNN-RNN architecture that takes a spectrogram as input [1, 6, 11]. It also closely resembles image captioning, where a CNN-RNN serves as the encoder in a sequence-to-sequence model [7, 10]. Following the same approach, we use a CNN-LSTM to encode the spectrogram and an LSTM decoder, followed by a fully connected layer, to

produce the desired outputs. Assuming an ocean environment, we train the model on synthetic acoustic pulses overlaid on real background noise.

II. THEORY

This section establishes the physical and methodological background required for the remainder of the report. We first describe the underwater acoustic environment in which the pulses of interest propagate. We then introduce the machine-learning components used to solve the task: the convolutional and recurrent building blocks, the sequence-to-sequence encoder-decoder framework that ties them together, and finally the metrics used to quantify performance.

A. The underwater acoustic environment

A hydrophone deployed in the ocean does not record a clean copy of an emitted pulse. The received signal is the emitted waveform after it has been attenuated by the medium, together with delayed echoes and ambient noise. The detector must recover the underlying pulses from this observation, and a realistic propagation model of the pulses in such an environment is therefore a prerequisite both for generating realistic training data and for understanding the limits of what can be inferred. We summarize the three effects that matter for this work: transmission loss, reverberation, and ambient noise, together with the signal types and the time-frequency representation that serves as the model input.

1. Transmission loss

As an acoustic wave travels outward from its source it loses intensity through two mechanisms. The first is geometric spreading: for a point source radiating into an unbounded medium the energy is distributed over a spherical wavefront whose area grows as r^2 , so the intensity falls as r^{-2} and the level decreases as $20 \log_{10} r$ relative to a reference range of 1 m. The second is absorption, the conversion of acoustic energy into heat through viscosity and chemical relaxation in seawater. Absorption is strongly frequency dependent, with high-frequency components attenuated far more rapidly than low-frequency ones. The total transmission loss in decibels is the sum of the two contributions,

$$\text{TL}(f, r) = \underbrace{20 \log_{10} r}_{\text{spherical spreading}} + \underbrace{\alpha(f) r}_{\text{absorption}}, \quad (1)$$

where r is the range in metres and $\alpha(f)$ is the absorption coefficient in dB/m. We use Thorp's empirical expression

for the absorption coefficient,

$$\alpha(f) = \frac{1}{1000} \left(3.3 \times 10^{-3} + \frac{0.11 f^2}{1 + f^2} + \frac{44 f^2}{4100 + f^2} + 3 \times 10^{-4} f^2 \right), \quad (2)$$

with f in kHz [8, 9].

2. Reverberation

The ocean is a waveguide bounded above by the surface and below by the seabed, and it contains scatterers such as marine life and suspended particles. A single emitted pulse therefore reaches the receiver not once but many times: the direct path arrives first, followed by a sequence of reflected and scattered replicas that have travelled longer paths. The persistence of received energy after the direct arrival is known as reverberation [9].

The delay of a given echo is set by the additional distance it travels. For an echo whose total path exceeds the direct path by Δr , the extra delay is

$$\Delta t = \frac{\Delta r}{c}, \quad (3)$$

where $c \approx 1500$ m/s is the speed of sound in seawater. The amplitude of each echo is reduced according to the transmission loss Eq. (1) evaluated over its longer path, so that successive echoes are progressively weaker. Reverberation is significant for the detection task because it produces multiple time-frequency signatures from a single physical pulse. A detector that has not learned to associate these replicas may report false additional pulses.

3. Ambient noise and signal types

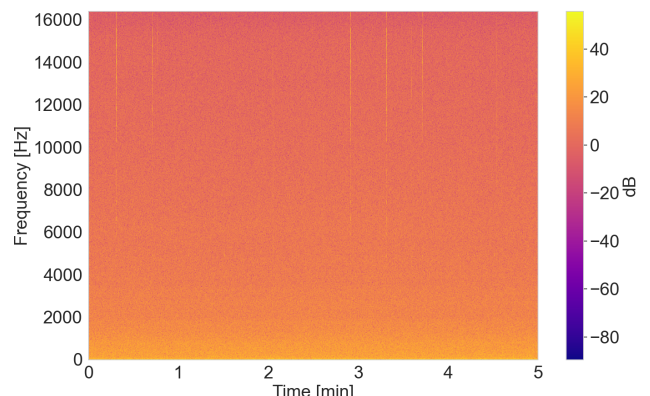


Figure 1: Spectrogram of a 5 minute recording from a bottom-mounted hydrophone [4].

Superimposed on the propagated pulses and their echoes is ambient ocean noise, arising from sources such as breaking waves, shipping, and biological activity. The background noise we will use is from a bottom-mounted hydrophone, shown in Fig. 1, that is part of NOAA’s passive acoustic data monitoring program [4].

The pulses we aim to detect and classify are short acoustic waves, distinguished by how their frequency behaves over the pulse duration: some hold a constant frequency, while others change frequency as they go. Each pulse is a frequency-swept cosine generated using `scipy.signal.chirp`, using the linear method for CW and LFM, and the hyperbolic method for HFM. Over a pulse of duration T sweeping from f_0 to f_1 , the instantaneous frequency $f(t)$ of the three types we consider is

$$\text{CW: } f(t) = f_0, \quad (4)$$

$$\text{LFM: } f(t) = f_0 + \frac{f_1 - f_0}{T} t, \quad (5)$$

$$\text{HFM: } f(t) = \frac{f_0 f_1 T}{(f_0 - f_1) t + f_1 T}. \quad (6)$$

Each pulse is thus characterized by its type together with four scalar quantities: its start and stop times and its start and end frequencies. These five attributes per pulse are precisely the quantities the model is trained to recover.

Lastly, throughout this work the strength of a pulse is expressed as a signal-to-noise ratio (SNR) in decibels relative to the root-mean-square level of the background, so that the noise sets the scale against which detectability is measured. The amplitude is then expressed as

$$A = \sigma_{rms} 10^{\text{SNR}/20}, \quad (7)$$

where σ_{rms} is the root-mean-square level of the background noise.

4. Time–frequency representation

Because the distinguishing feature of each pulse type is the evolution of its frequency content over time, a representation that resolves both time and frequency is required. The short-time Fourier transform (STFT) provides this by tracking how the signal’s frequency content changes across short successive time windows, giving at each time τ and frequency f a value $X(\tau, f)$ whose magnitude is the strength of that frequency at that time. Its squared magnitude as a function of time and frequency is the spectrogram, which we express in decibels,

$$S(\tau, f) = 20 \log_{10} (|X(\tau, f)| + \epsilon), \quad (8)$$

where ϵ is a small constant added to avoid the singularity of the logarithm at zero. The resulting two-dimensional, image-like representation is the input to the network.

B. Convolutional neural networks

The spectrogram is an image-like array in which the pulses are confined to small regions of the time–frequency plane. Convolutional neural networks (CNNs) are well suited to such data because they extract features through small learnable filters that are convolved across the input. For a two-dimensional input I and kernel K , the discrete convolution is

$$(I * K)[i, j] = \sum_m \sum_n I[i - m, j - n] K[m, n]. \quad (9)$$

This operation suits spectrograms well. Each filter scans the whole input for a small local pattern, detecting it wherever it occurs. A convolutional layer learns many such filters, and stacking layers lets each one build on the patterns found before it, until the full pulse is detected. [5].

Each convolutional layer in this work is followed by three standard components. Batch normalisation rescales the activations within each mini-batch, which accelerates and stabilises training. A nonlinear activation function, the exponential linear unit (ELU), introduces the nonlinearity required to model complex mappings. Finally, dropout randomly zeroes a fraction of activations during training, acting as a regularizer that discourages the network from over-relying on any single feature [5].

C. Recurrent neural networks and the LSTM

Whereas a CNN processes a fixed grid, the output of our task is a *sequence* of pulses. A recurrent neural network (RNN) processes a sequence one step at a time, passing a hidden state forward so that earlier elements can influence succeeding ones. Plain RNNs struggle to retain information over long sequences however, as the the gradients of the loss during backpropagation tends to vanish or explode. The long short-term memory (LSTM) network solves this with a set of gates that control what to keep, update, and output at each step, letting it carry relevant information over many steps. This is what allows the decoder to keep track of how many pulses it has emitted and what their attributes were [5].

A standard LSTM runs in one direction, so its state at each step reflects only the past. For a complete spectrogram the whole input is available at once, and both directions are informative. A bidirectional LSTM runs one pass forward and one backward and combines them, so each position is represented using context from both sides. We use this in the encoder to compress the whole spectrogram into a representation that captures every pulse present [5].

D. Sequence-to-sequence encoder-decoder models

The central difficulty of the task is that the output has no fixed size: a given spectrogram may contain anywhere from zero to several pulses, and the number is not known in advance. The sequence-to-sequence (seq2seq) encoder-decoder framework [7] is built precisely for mappings between sequences of differing and variable length, and it is the framework we adopt.

The model consists of two parts. The *encoder*, which is a CNN followed by a bidirectional LSTM, reads the entire input and compresses it into a fixed-size context vector, which is given by the final hidden and cell states of the LSTM. The *decoder* is a unidirectional LSTM that emits the pulses in sequence and is initialized by the context vector. Note that the output at each step of the decoder is fed to five fully connected layers: one classifier that determines pulse type and four regressors that determine start and end frequency and time.

1. Variable-length output and the EOS token

For the decoder to produce sequences of variable length, it needs a mechanism to signal that it has finished. This is provided by a dedicated end-of-sequence (EOS) token: a special symbol appended to every target sequence during training, which the decoder learns to emit once all genuine pulses have been produced. At inference the decoder generates pulses until it predicts an EOS (or until a maximum length is reached), and the predicted EOS thus determines the number of pulses returned. An empty spectrogram is represented by a target consisting of the EOS token alone. In this work the token is an eight-element vector whose first four entries are a one-hot classification over [CW, LFM, HFM, EOS] and whose remaining four entries hold the start time, stop time, start frequency, and end frequency of the pulse.

2. Teacher forcing

The decoder is autoregressive: each step takes the previous step's output as part of its input. During training this creates a choice. One option is to feed the decoder its own predictions. However, early in training, these predictions are poor, and errors compound along the sequence which makes learning slow and unstable. The alternative, known as *teacher forcing*, is to feed the decoder the ground-truth token from the previous position instead of its own prediction, so that each step is conditioned on correct history regardless of earlier mistakes. We train with teacher forcing and generate autoregressively at inference [2].

3. Multi-task output and loss components

Each pulse carries information of two different kinds, a discrete type and four continuous time-frequency values. The decoder must therefore have several parallel heads sharing the decoder state: one classification head, which produces scores over the four classes, and four regression heads, one for each scalar quantity. Training such a multi-task model requires a loss with two corresponding parts. The classification head is trained with the cross-entropy loss, which for a single position with true class c and predicted class probabilities p_k is

$$\mathcal{L}_{\text{class}} = -\log p_c, \quad (10)$$

while each regression head is trained with the mean squared error between its prediction $\hat{y}$ and the target y ,

$$\mathcal{L}_{\text{reg}} = (\hat{y} - y)^2. \quad (11)$$

The total training objective is a weighted combination of the classification and regression terms.

E. Evaluation metrics

Evaluating the model is less straightforward compared to a fixed-output task. This is because the predicted and true pulses may differ in number, and the model does not label which prediction belongs to which true pulse, so the two must be paired up before they can be compared. We therefore evaluate in two stages: a matching stage that pairs predicted and true pulses, followed by the computation of detection, classification, and regression metrics on the result.

1. Matching

Predicted and true pulses are paired one-to-one by proximity in start time, with the closest pairs matched first and any pair separated by more than a fixed tolerance are left unmatched. Each matched pair counts as a true positive (TP), each predicted pulse left unmatched is a false positive (FP), or false alarm, and each true pulse left unmatched is a false negative (FN), or miss.

2. Detection

The quality of detection, independent of how accurately attributes are estimated, is summarised by precision, recall, and the F_1 score:

$$\begin{aligned} \text{Precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}}, \\ \text{Recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}}, \\ F_1 &= 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \end{aligned} \quad (12)$$

Precision measures how many detected pulses are real, recall how many real pulses are detected, and F_1 balances the two.

3. Classification

For correctly detected pulses (the matched pairs), the pulse type is scored by its classification accuracy,

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{matched}}}, \quad (13)$$

where N_{correct} is the number of matched pulses classified correctly and N_{matched} is the total number of matched pulses.

4. Regression

For each matched pair, the error in each of the four continuous quantities is the signed difference between prediction and truth. Over all matched pulses we summarize these errors by their mean, which serves as a measure of the typical bias in each predicted quantity.

5. Count

Finally, since the number of pulses is itself a prediction, we report count-level metrics, comparing the predicted and true number of pulses per spectrogram.

III. METHOD

This section describes how the data were generated, how the model is constructed, and how it was trained and evaluated.

A. Dataset generation

1. Background noise

The background for every example is real ocean noise, taken from the bottom-mounted hydrophone recording described in Sec. II A 3. The recording is resampled to a sampling frequency of 30 kHz and divided into non-overlapping segments of 5 s. Each segment is normalized independently to zero mean and unit variance (a z -score normalization), so that the synthetic pulses are added to a consistent noise floor across segments.

2. Synthetic pulses

Two kinds of example are generated. Noise-only examples consist of a normalized segment with no added

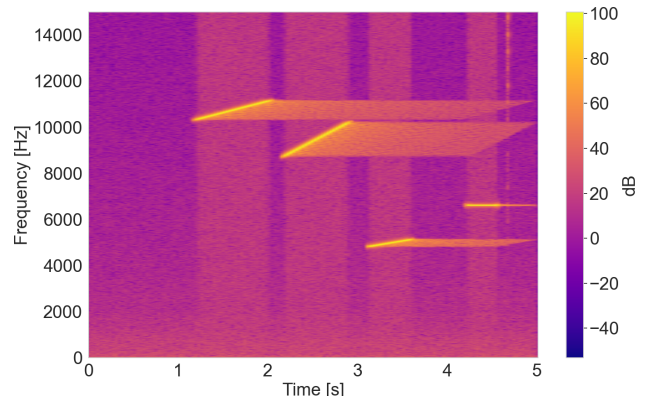


Figure 2: Example spectrogram of a 5 s segment of ocean noise with several synthetic pulses added. The magnitude is shown in decibels.

pulse. Pulse examples are formed by adding between one and four pulses to a randomly drawn segment. The pulse start and stop times are sampled so that the pulses do not overlap in time and fit within the 5 s window, with each pulse lasting between 0.15 s and 1.5 s. Each pulse is independently assigned a type (CW, LFM, or HFM), a start and end frequency in the range 1000 Hz to 15 000 Hz, a signal-to-noise ratio, and a source distance. The amplitude is set from the chosen SNR through Eq. (7).

The propagation effects of Sec. II A are then applied. Each pulse is attenuated by the transmission loss over its path [Eqs. (1) and (2)], with source distances drawn from 10 m to 50 m, and the direct arrival is followed by a train of reverberation echoes. Assuming a sound speed of 1500 m/s, each echo is delayed according to its additional path length [Eq. (3)] and attenuated by the transmission loss over that longer path. Finally, every pulse and echo is shaped by a Tukey-window before being added to the noise segment.

3. Spectrogram representation

Each example is transformed into the time-frequency representation of Sec. II A 4. The short-time Fourier transform is computed with a segment length of $2^{11} = 2048$ samples and 50% overlap, and its magnitude is converted to decibels following Eq. (8). The result is a single spectrogram image per example, which forms the input to the network. A representative example containing several pulses is shown in Fig. 2.

4. Target encoding

The target for each example is a sequence of pulses, with each pulse encoded as the eight-element vector in-

troduced in Sec. IID1,

$$\left[\underbrace{\text{CW, LFM, HFM, EOS}}_{\text{one-hot encoded}}, t_{\text{start}}, t_{\text{stop}}, f_1, f_2 \right], \quad (14)$$

whose first four entries are a one-hot encoded and whose last four hold the start time, stop time, start frequency, and end frequency. Every target sequence is terminated by an end-of-sequence (EOS) token, the vector with only the EOS flag set, and the rest as zero. A noise-only example is represented by the EOS token alone. Because the examples contain different numbers of pulses, the sequences are padded with zero vectors to a common length of ten so they can be stacked into a single batch tensor. As each example contains at most four pulses plus the terminating EOS token, this length is really never exceeded, but is useful to have if it were to be exceeded in a realistic setting.

Lastly, the two regression quantities are rescaled to lie in a comparable range before training. The times are divided by the 5 s window length and the frequencies by 15 000 Hz, so that both fall in $[0, 1]$. This prevents the much larger numerical range of the frequencies from dominating the regression loss.

5. Dataset size and split

A total of 30000 examples were generated and divided into a training set (67%) and a test set (33%) using a fixed random seed. The test set is used both to monitor training and to report final performance, as described in Sec. IIIC.

B. Model architecture

The model follows the sequence-to-sequence encoder-decoder design of Sec. IID. The encoder consists of a CNN followed by a bidirectional LSTM. The decoder is a unidirectional LSTM terminating in the multiple output heads discussed above. The full layer-by-layer specification is given in Table I.

The convolutional encoder is a stack of six convolutional blocks. Each block applies a convolution with a 3×3 kernel, batch normalization, an ELU activation, and dropout, progressively increasing the number of feature maps from 16 in the first block to 512 in the last while reducing the frequency dimension to limit computational cost. The resulting feature maps are then reshaped into a sequence along the time axis. The feature columns belonging to each time step are flattened into a single vector, giving one vector per step. This sequence is passed to the bidirectional LSTM encoder, whose final forward and backward hidden and cell states are concatenated into a context vector that initializes the decoder.

Table I: Layer-by-layer specification of the model. The convolutional blocks each comprise a 3×3 convolution followed by batch normalisation, an ELU activation, and dropout ($p = 0.1$), all use stride $(2, 1)$ so that only the frequency dimension is downsampled. The decoder heads each map the decoder hidden state to their respective outputs.

Component	Layer	Output channels / size
CNN encoder	Conv block 1	16
	Conv block 2	32
	Conv block 3	64
	Conv block 4	128
	Conv block 5	256
	Conv block 6	512
Reshape	to sequence	$(T, 512 \times 15)$
LSTM encoder	Bidirectional LSTM	hidden 512
	layers	2
LSTM decoder	Unidir. LSTM	hidden 1024
	layers	2
Output heads	Classification	4
	Start time	1
	Stop time	1
	Start freq.	1
	End freq.	1

At each step the decoder produces, from its hidden state, a classification head over the four token classes and four regression heads, one for each of the start time, stop time, start frequency, and end frequency.

C. Training and evaluation

1. Loss function

The model is trained by minimizing a combined loss built from the cross-entropy and mean-squared-error terms of Sec. IID3. Since predictions are made at every step of a padded sequence, the losses must be restricted to the appropriate positions. A target sequence consists of the real pulses, the EOS token that terminates it, and the zero-padding that fills the remaining steps. The classification loss is taken over the real pulses and the EOS, so that the model learns both to identify the pulse types and to mark where the sequence ends. The regression loss is taken over the real pulses alone, since the time and frequency entries carry no meaning at the EOS and padding positions.

Each component is averaged over the positions it covers, and the total loss is their weighted sum,

$$\mathcal{L} = \mathcal{L}_{\text{class}} + \lambda \mathcal{L}_{\text{reg}}, \quad (15)$$

where $\mathcal{L}_{\text{reg}}$ is the sum of the mean-squared errors of the four regression heads. The regression term is given the larger weight $\lambda = 10$, so that the four continuous targets, scaled to $[0, 1]$, are not overwhelmed by the classification term.

2. Optimization

During training the decoder is run with teacher forcing, i.e., at each step it receives the ground-truth token from the previous position rather than its own prediction. The model is trained with the Adam optimizer at a learning rate of 10^{-3} , a batch size of 32, and gradient-norm clipping at 1.0 to stabilize training, for up to 50 epochs. After each epoch the loss on the held-out set is computed, and the checkpoint with the lowest held-out loss is retained for evaluation. This acts as a model-selection criterion that guards against overfitting.

3. Inference and evaluation

At inference the decoder runs autoregressively, feeding its own prediction back as the next input and generating pulses until it emits an EOS or reaches the maximum length. The predicted time and frequency values are rescaled back to physical units by inverting the scaling applied during training.

The model is evaluated on the held-out set using the two-stage protocol of Sec. II E. The predicted and true pulses are first matched by proximity in start time, and detection, classification, and regression metrics are then computed on the result. We note that the test set is used both to choose which checkpoint to keep and to report the final metrics. Since the checkpoint was selected partly because it performed well on this set, the reported scores are likely a little optimistic compared to truly unseen data. A fully separate test set would avoid this and is the cleaner choice for future work.

IV. RESULTS

This section reports the performance of the trained model. We first describe the training behavior and the selection of the evaluated checkpoint, and then present the evaluation on the test set.

A. Training behaviour

Figure 3 shows the training and validation loss over the 50 epochs. Both fall sharply over the first few epochs and then continue to decrease more slowly. After about epoch 20 the two curves diverge: the training loss keeps falling while the validation loss begins to rise, the characteristic signature of overfitting. The retained checkpoint is the one at the validation minimum, at epoch 20 with a validation loss of 0.3393 (marked in the figure). All results that follow use this checkpoint.

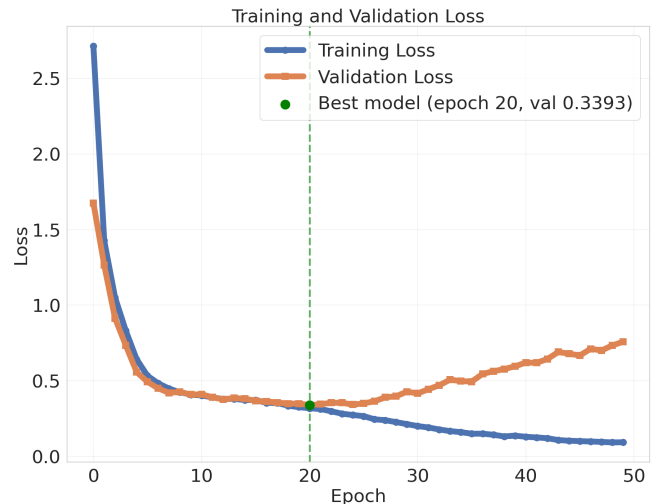


Figure 3: Training and validation loss over the 50 training epochs. The validation loss reaches its minimum at epoch 20.

B. Pulse count

Figure 4 shows the confusion matrix between the true and predicted number of pulses per spectrogram. The matrix is almost perfectly diagonal. The model recovers the exact number of pulses in essentially every case, including the noise-only examples. The only departures are a handful that are off by one.

C. Detection

After the pulse predictions are matched to the true pulses, the detection outcomes are summarized in Fig. 5. Of the matched and unmatched pulses there are 24 639 true positives, against only 52 false positives (false alarms) and 67 false negatives (misses). This corresponds to a precision of 0.998, a recall of 0.997, and an F_1 score of 0.998. The model thus detects essentially every pulse present while raising very few false alarms.

D. Classification

The accuracy of the predicted pulse type, computed over the matched pulses, is shown as a confusion matrix in Fig. 6. The behaviour differs sharply between the constant-frequency and the frequency-modulated types. CW pulses are classified almost perfectly, with 99% assigned the correct type and negligible confusion with either swept type. The two swept types, however, are frequently confused with one another. LFM pulses are correctly identified only 59% of the time, with the remaining 41% assigned to HFM, and HFM pulses are correct 63% of the time, with 37% assigned to LFM. Confusion be-

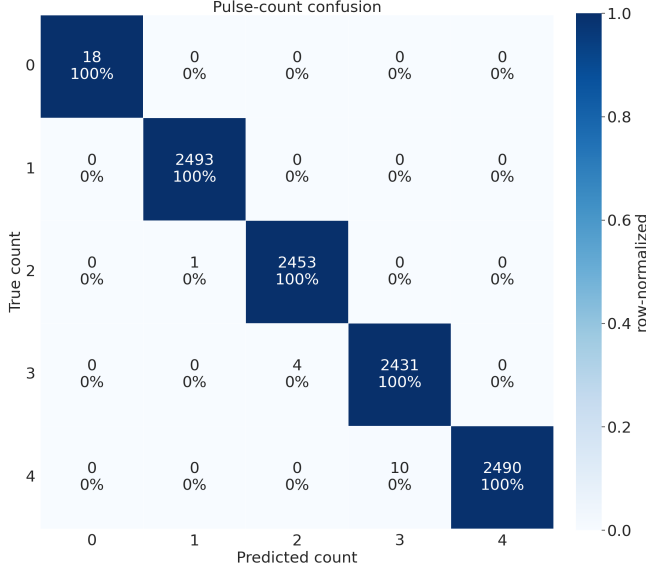


Figure 4: Confusion matrix between the true and predicted number of pulses per spectrogram. Cell entries give the example count and the row-normalized percentage. The near-perfect diagonal indicates that the predicted pulse count matches the true count in almost all cases.

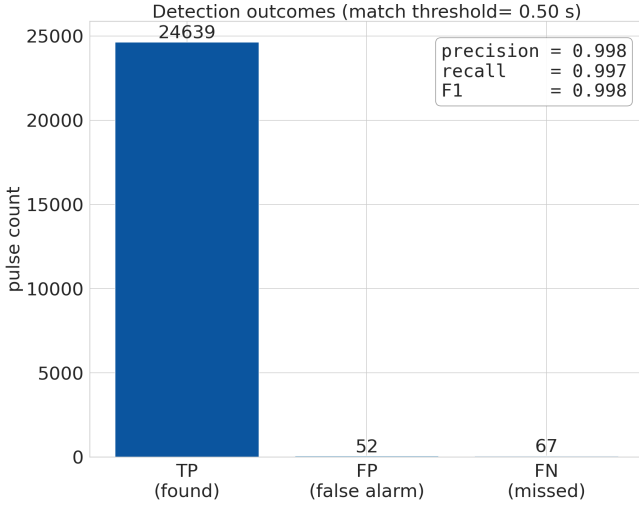


Figure 5: Detection outcomes after matching, using a start-time match threshold of 0.5 s. The bars show the number of true positives (found), false positives (false alarms), and false negatives (misses). The precision, recall, and F_1 metrics are shown in the legend.

tween a swept type and CW is rare in both directions. The distinction between the two frequency-modulated types is therefore the principal source of classification error.

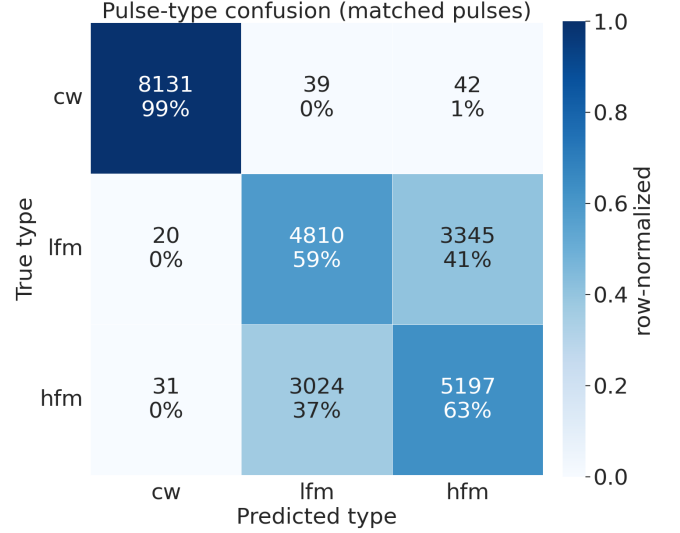


Figure 6: Confusion matrix of the predicted pulse type over the matched pulses. Cell entries give the pulse count and the row-normalized percentage. CW is classified almost perfectly, whereas LFM and HFM are frequently confused with each other.

E. Regression

The accuracy of the four estimated quantities, the start and stop times and the start and end frequencies, is shown in Figs. 7 and 8, evaluated over the matched pulses. Figure 7 shows the predicted value against the true value for each quantity. The start and end time lie tightly along the $y = x$ line across the full 5 s window, with only minor scatter. The two frequencies also follow the diagonal across the 1000 Hz to 15 000 Hz range, though with a wider scatter and a number of outliers that depart substantially from the true value.

The corresponding distributions of the signed error (predicted minus true) are shown in Fig. 8. All four are sharply peaked and centred close to zero. The mean errors are -0.002 s for the start time and $+0.027$ s for the stop time, both small fractions of the typical pulse duration, and -17 Hz and $+81$ Hz for the start and end frequencies, both small relative to the 15 000 Hz range. The estimates are therefore accurate and essentially unbiased, with the wider spread of the frequency errors consistent with the greater scatter seen in Fig. 7.

V. DISCUSSION

The results show that the model performs the detection and localization parts of the task almost perfectly. It recovers the correct number of pulses in nearly every spectrogram, detects essentially every pulse with very few

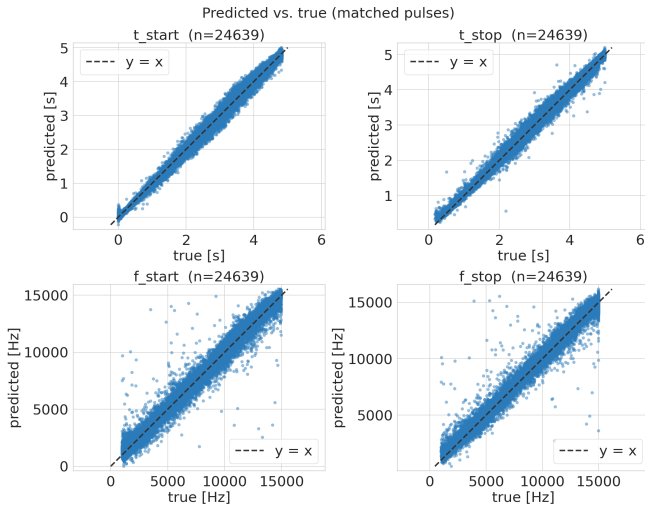


Figure 7: Predicted versus true values over the matched pulses, for the start time, stop time, start frequency, and end frequency. The dashed line is $y = x$. The time estimates lie tightly on the diagonal. The frequency estimates follow it with a wider scatter and some outliers.

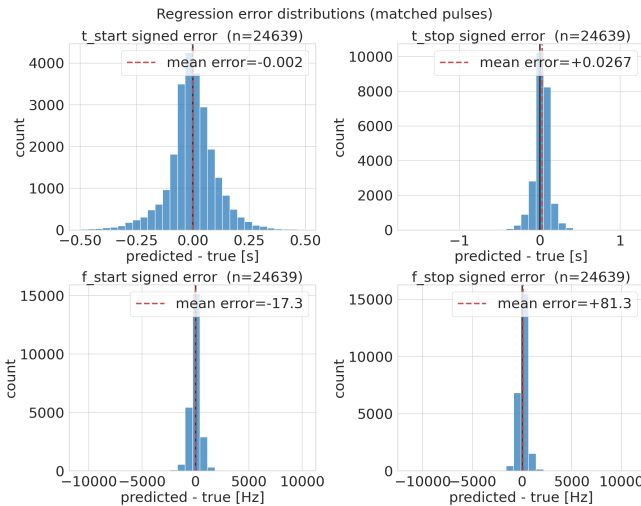


Figure 8: Distributions of the signed error (predicted minus true) over the matched pulses, for the start time, stop time, start frequency, and end frequency. The dashed line marks the mean error in each panel. All four distributions are sharply peaked and centered close to zero.

false alarms, and estimates the pulse times and frequencies with small, largely unbiased errors. It also classifies CW pulses almost perfectly. The single substantial weakness is the confusion between the two frequency-modulated types, LFM and HFM. The remainder of this section examines that weakness, the behavior of the regression estimates, and the limitations of the study.

A. Confusion between the swept types

The classification behavior separates cleanly into the constant-frequency type and the two swept types. CW pulses appear on the spectrogram as flat horizontal ridges, visually distinct from any sweep, and are correctly identified almost without exception. The distinction between LFM and HFM is far harder, and is the dominant source of classification error.

A plausible explanation is that, over a limited frequency range, a hyperbolic sweep is close to a linear one. The hyperbola in Eq. (6) is locally well approximated by a straight line, so the two sweep types trace nearly the same ridge on the spectrogram and differ only in a slight curvature that is easily lost in noise and reverberation. The short pulse durations used here (0.15 s to 1.5 s) leave little time for the curvature to develop, making the two types genuinely difficult to separate from the time-frequency image alone. The near-symmetry of the confusion, with LFM called HFM 41% of the time and HFM called LFM 37% of the time, is consistent with this picture. The model is not systematically defaulting to one class but is faced with two classes that are intrinsically hard to tell apart.

B. Detection and localization

The strength of the model on detection, counting, and localization can be understood from the structure of the task. A pulse appears in the spectrogram as a localized region of high energy against the noise background, which is exactly the kind of local structure the convolutional encoder is suited to extract, and its temporal extent is directly visible. The sequence-to-sequence decoder, in turn, handles the variable number of pulses naturally through the EOS mechanism, which the near-perfect pulse-count matrix confirms.

The estimated times are more accurate than the estimated frequencies, which show a wider scatter and a number of outliers in Fig. 7. A likely contributor is the asymmetry built into the encoder. The convolutional blocks use a stride of (2,1), downsampling the frequency axis at every block while leaving the time axis intact, so that frequency is ultimately represented at a much coarser resolution than time. The network therefore reads time more finely than frequency, consistent with the tighter time estimates.

C. Limitations

Several limitations should be kept in mind when interpreting these results.

The most important is that the model is trained and evaluated entirely on synthetic data. The pulses are generated from idealized models of propagation, the transmission loss of Eqs. (1) and (2) and a simplified rever-

beration train, added to recorded noise. Real recordings would contain effects this generative process does not capture, such as more complex multi-path structure, non-stationary noise, and variability in propagation conditions. The model has therefore only been exposed to its own generative assumptions, and its performance on genuine recordings remains to be established. The data generator also places the pulses so that they never overlap in time. In realistic scenes pulses can arrive simultaneously or overlap, and the model has not been trained to separate such cases. The frequency band, the pulse-count ceiling of four, and the 5 s window are likewise fixed, so performance outside these ranges is untested. Finally, as noted in Sec. III C, the test set is used both to select the retained checkpoint and to report the final metrics. No information leaks into the weights, since the model is never trained on this set, but the reported scores are likely to be slightly optimistic compared with truly unseen data.

D. Future work

The natural next step is to address the LFM–HFM confusion. If the difficulty is the weak curvature of short, narrowband sweeps, then training and testing on longer or wider-band pulses, where the hyperbolic and linear shapes diverge more clearly, would test that explanation directly. A finer frequency representation, obtained by downsampling the frequency axis less aggressively in the encoder, may also help the model separate the two types, and could at the same time reduce the scatter in the frequency estimates.

Beyond this, applying the model to real labeled recordings, or adapting it from synthetic to real data, would establish its practical value, and extending the data generator to include overlapping pulses would broaden its

applicability to realistic scenes. A fully separate test set, distinct from the data used for checkpoint selection, would also remove the mild optimism in the reported metrics.

VI. CONCLUSION

We have presented a CNN–LSTM encoder–decoder that detects acoustic pulses in a spectrogram and jointly estimates their type, timing, and frequency content, and is trained on synthetic sonar pulses overlaid on real ocean noise.

On synthetic test data the model is strong on every part of the task but one. It recovers the correct number of pulses in almost all cases, detects essentially every pulse with very few false alarms ($F_1 = 0.998$), and estimates the start and stop times and the start and end frequencies with small and largely unbiased errors. Continuous-wave pulses are classified almost perfectly. The single substantial weakness is the discrimination between the two frequency-modulated types. Linear and hyperbolic sweeps are frequently confused with one another, which we attribute to the near-linearity of a short, narrowband hyperbolic sweep, leaving the two types nearly indistinguishable in the time–frequency image.

These results suggest that the joint detection-and-localization framing is well matched to the architecture, and that the remaining challenge is the finer discrimination of sweep shape. The most promising directions are therefore to make this distinction easier to learn, through longer or wider-band pulses or a finer frequency representation, and to move from synthetic toward real recordings, including the overlapping pulses that occur in realistic scenes. With these extensions, the approach offers a compact and fully learned route to characterizing acoustic pulses directly from their spectrogram.

-
- [1] K. Choi, G. Fazekas, M. Sandler, and K. Cho. Convolutional recurrent neural networks for music classification. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 2392–2396, 2017.
 - [2] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
 - [3] N. Levanon and E. Mozeson. *Radar Signals*. John Wiley & Sons, Ltd, 2004.
 - [4] NOAA National Centers for Environmental Information. Passive Acoustic Data Collection, 2017. Accessed: June 7, 2026.
 - [5] S. Raschka, Y. Liu, and V. Mirjalili. *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing Ltd., 2022.
 - [6] T. N. Sainath, O. Vinyals, A. Senior, and H. Sak. Convolutional, Long Short-Term Memory, fully connected Deep Neural Networks. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4580–4584, 2015.
 - [7] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks, 2014.
 - [8] W. H. Thorp. Analytic description of the low-frequency attenuation coefficient. *The Journal of the Acoustical Society of America*, 42(1):270–270, 07 1967.
 - [9] R. Urick. *Principles of Underwater Sound*. McGraw-Hill, 1983.
 - [10] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan. Show and Tell: A Neural Image Caption Generator. *CoRR*, abs/1411.4555, 2014.
 - [11] Y. Zhang, Q. Huang, W. Sun, F. Chen, D. Lin, and F. Chen. Research on lung sound classification model based on dual-channel CNN-LSTM algorithm. *Biomedical Signal Processing and Control*, 94:106257, 2024.